

5. Strings

Contents

Trie (s)	1
Global Arrays	1
Unordered_map	2
Forward Start Trie	2
Rolling Hash	3
KMP	5
Z-Algo	6
Largest Lexicographical Substring	6
Manacher	6
Aho Corasick Algorithm	7
Dynamic Aho Corasick	8
Suffix Array	11
Suffix Automaton	12

Trie (s)

Global Arrays

```
1  const int MXN = 1000005, S = 26, OFFSET = 'a';
2  int nxt[MXN][S], cnt[MXN], isend[MXN], nodes = 1;
3  // clear the arrays you add here in clear() function
4  void clear() {
5      for (int i = 0; i < nodes; i++) {
6          cnt[i] = isend[i] = 0;
7          memset(nxt[i], 0, sizeof(nxt[i]));
8      }
9      nodes = 1;
10 }
11 void insert(const string& s) {
12     int u = 0;
13     for (char c : s) {
14         int v = c - OFFSET;
15         if (!nxt[u][v]) nxt[u][v] = nodes++;
16         u = nxt[u][v];
17         cnt[u]++;
18     }
19     isend[u]++;
```

```

20 }
21 int search(const string& s) {
22     int u = 0;
23     for (char c : s) {
24         int v = c - OFFSET;
25         if (!nxt[u][v]) return false;
26         u = nxt[u][v];
27     }
28     return cnt[u];
29 }
30 // clear() insert()

```

Unordered_map

```

1  const int MXN = 1000005;
2  struct Node {
3      unordered_map<char, int> nxt;
4      int cnt = 0;
5  } tree[MXN];
6  int node_cnt = 1;
7  void clear() {
8      for (int i = 0; i < node_cnt; i++) {
9          tree[i].nxt.clear();
10         tree[i].cnt = 0;
11     }
12     node_cnt = 1;
13 }
14 void insert(const string& s) {
15     int u = 0;
16     for (char c : s) {
17         auto it = tree[u].nxt.find(c);
18         if (it == tree[u].nxt.end()) {
19             tree[u].nxt[c] = node_cnt;
20             u = node_cnt++;
21         } else {
22             u = it->second;
23         }
24         tree[u].cnt++;
25     }
26 }
27 int search(const string& s) {
28     int u = 0;
29     for (char c : s) {
30         auto it = tree[u].nxt.find(c);
31         if (it == tree[u].nxt.end()) return false;
32         u = it->second;
33     }
34     return tree[u].cnt;
35 }
36 // clear() insert()

```

Forward Start Trie

```

1  const int MXN = 1000005;
2  int head[MXN]; // points to the first child of a node
3  int nxt[MXN]; // points to the next sibling of a node

```

```

4 char val[MXN]; // the character that leads to this node
5 int cnt[MXN], isend[MXN];
6 int node_cnt = 1;
7 void clear() {
8     fill(head, head + node_cnt, 0);
9     fill(nxt, nxt + node_cnt, 0);
10    fill(cnt, cnt + node_cnt, 0);
11    fill(isend, isend + node_cnt, 0);
12    node_cnt = 1;
13 }
14 void insert(const string& s) {
15     int u = 0;
16     for (char c : s) {
17         int child = head[u];
18         bool found = false;
19         while (child) {
20             if (val[child] == c) {
21                 u = child;
22                 found = true;
23                 break;
24             }
25             child = nxt[child];
26         }
27         if (!found) {
28             val[node_cnt] = c;
29             nxt[node_cnt] = head[u];
30             head[u] = node_cnt;
31             u = node_cnt++;
32         }
33         cnt[u]++;
34     }
35 }
36 int search(const string& s) {
37     int u = 0;
38     for (char c : s) {
39         int child = head[u];
40         bool found = false;
41         while (child) {
42             if (val[child] == c) {
43                 u = child;
44                 found = true;
45                 break;
46             }
47             child = nxt[child];
48         }
49         if (!found) return 0;
50     }
51     return cnt[u];
52 }

```

Rolling Hash

```

1 using u64 = uint64_t;
2 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count() ^
3     (uintptr_t)make_unique<char>().get());
4

```

```

5 struct hash61 {
6     static const u64 md = (1LL << 61) - 1;
7     inline static u64 step = (md >> 2) + rng() % (md >> 1);
8     inline static vector<u64> pw = {1};
9
10    int n;
11    vector<u64> pref, suff;
12
13    u64 add(u64 a, u64 b) const { return (a += b) >= md ? a - md : a; }
14    u64 sub(u64 a, u64 b) const { return (a += md - b) >= md ? a - md : a; }
15    u64 mul(u64 a, u64 b) const { return __uint128_t(a) * b % md; }
16
17    template<class T>
18    hash61(const T& s) : n(s.size()), pref(n + 1), suff(n + 1) {
19        while (pw.size() <= n) pw.push_back(mul(pw.back(), step));
20        pref[0] = suff[n] = 1;
21        for (int i = 0; i < n; i++) pref[i + 1] =
22            add(mul(pref[i], step), s[i]);
23        for (int i = n - 1; i >= 0; i--) suff[i] =
24            add(mul(suff[i + 1], step), s[i]);
25    }
26
27    u64 operator()(int l, int r) const { return sub(pref[r + 1],
28        mul(pref[l], pw[r - l + 1])); }
29    u64 rev(int l, int r) const { return sub(suff[l],
30        mul(suff[r + 1], pw[r - l + 1])); }
31 };
32
33 struct hash61 {
34     static const u64 md = (1LL << 61) - 1;
35     inline static u64 step = (md >> 2) + rng() % (md >> 1);
36
37     inline static u64 power(u64 b, u64 e) {
38         u64 r = 1;
39         while (e) {
40             if (e & 1) r = (unsigned __int128)r * b % md;
41             b = (unsigned __int128)b * b % md;
42             e >>= 1;
43         }
44         return r;
45     }
46
47     inline static u64 inv_step = power(step, md - 2);
48     inline static vector<u64> pw = {1}, ipw = {1};
49
50     int n = 0;
51     vector<u64> pref = {1}, suff = {0};
52
53     u64 add(u64 a, u64 b) const { return (a += b) >= md ? a - md : a; }
54     u64 sub(u64 a, u64 b) const { return (a += md - b) >= md ? a - md : a; }
55     u64 mul(u64 a, u64 b) const { return (unsigned __int128)a * b % md; }
56
57     hash61() {}
58     template<class T> hash61(const T& s) { for (auto c : s) push_back(c); }
59
60     void push_back(u64 c) {

```

```

61     while (pw.size() <= n + 1) {
62         pw.push_back(mul(pw.back(), step));
63         ipw.push_back(mul(ipw.back(), inv_step));
64     }
65     pref.push_back(add(mul(pref.back(), step), c));
66     suff.push_back(add(suff.back(), mul(c, pw[n])));
67     n++;
68 }
69
70 void pop_back() {
71     if (!n) return;
72     pref.pop_back();
73     suff.pop_back();
74     n--;
75 }
76
77 u64 operator()(int l, int r) const { return sub(pref[r + 1], mul(pref[l],
78     pw[r - l + 1])); }
79 u64 rev(int l, int r) const { return mul(sub(suff[r + 1], suff[l]), ipw[l]); }
80 };

```

KMP

```

1  vector<int> KMP(const string &s){
2      int n = s.length();
3      vector<int> pi(n);
4      for (int i = 1; i < n; i++){
5          int j = pi[i - 1];
6          while (j > 0 && s[i] != s[j]){
7              j = pi[j - 1];
8          }
9          if (s[i] == s[j]) j++;
10         pi[i] = j;
11     }
12     return pi;
13 }
14 vector<int> find_occurrences(const string &pat, const string &text){
15     string s = pat + '#' + text;
16     vector<int> pi = KMP(s), ret;
17     int m = pat.size();
18     for (int i = m + 1; i < (int)s.size(); i++){
19         if (pi[i] == m) ret.push_back(i - 2 * m);
20     }
21     return ret;
22 }
23 vector<vector<int>> automaton(string s){ // aut[s.length() + 1][26]
24     s += '#';
25     int n = s.length();
26     vector<int> pi = KMP(s);
27     vector<vector<int>> aut(n, vector<int>(26));
28     for (int i = 0; i < n; i++){
29         for (int j = 0; j < 26; j++){
30             if (i > 0 && 'a' + j != s[i]){
31                 aut[i][j] = aut[pi[i - 1]][j];
32             }else{

```

```

33         aut[i][j] = i + ('a' + j == s[i]);
34     }
35 }
36 }
37 return aut;
38 }

```

Z-Algo

```

1  vector<int> z_algo(string s) {
2      vector<int> z(s.size());
3      for(int i = 1, l = 0, r = 0; i < s.size(); i++) {
4          if(i < r) z[i] = min(r - i, z[i - l]);
5          while(i + z[i] < s.size() && s[z[i]] == s[z[i] + i]) z[i]++;
6          if(i + z[i] > r) r = i + z[i], l = i;
7      }
8      return z;
9  }

```

Largest Lexicographical Substring

```

1  string largestLexSubstring(const string &s) {
2      int n = int(s.size());
3      int i = 0, j = 1, k = 0;
4
5      while (j + k < n) {
6          if (s[i + k] == s[j + k]) k++;
7          else if (s[i + k] < s[j + k])
8              /* change it to > if you want lowest */
9              i = max(i + k + 1, j), j = i + 1, k = 0;
10         else j = j + k + 1, k = 0;
11     }
12
13     return s.substr(i);
14 }

```

Manacher

```

1  auto manacher(const string &t) {
2      string s = "%#";
3      s.reserve(t.size() * 2 + 3);
4      for(char c : t) s += c + "#";
5      s += '$';
6      // t = aabaacaabaa -> s = %a#a#b#a#a#c#a#a#b#a#a#$
7
8      vector<int> res(s.size());
9      for(int i = 1, l = 1, r = 1; i < s.size(); i++) {
10         res[i] = max(0, min(r - i, res[l + r - i]));
11         while(s[i + res[i]] == s[i - res[i]]) res[i]++;
12         if(i + res[i] > r) {
13             l = i - res[i];
14             r = i + res[i];

```

```

15     }
16 }
17 for(auto &i : res) i--;
18 return vector(res.begin() + 2, res.end() - 2); // a#a#b#a#a#c#a#a#b#a#a
19 // get max odd len = res[2 * i]; aba -> i = b
20 // get max even len = res[2 * i + 1]; abba -> i = first b
21 }

```

Aho Corasick Algorithm

```

1 namespace corasick {
2     const int N = +++++;
3     const int SIGMA = +++++;
4
5     int nxt[N][SIGMA], fail_link[N], dict_link[N], match_idx[N], nodes;
6
7     void init() {
8         nodes = fail_link[0] = dict_link[0] = 0;
9         memset(nxt[0], 0, sizeof(nxt[0]));
10        match_idx[0] = -1;
11        nodes++;
12    }
13
14    int create_node() {
15        memset(nxt[nodes], 0, sizeof(nxt[nodes]));
16        fail_link[nodes] = dict_link[nodes] = 0;
17        match_idx[nodes] = -1;
18        return nodes++;
19    }
20
21    int insert(const string& pattern, int id) {
22        int cur = 0;
23        for (char c : pattern) {
24            if (!nxt[cur][c]) {
25                nxt[cur][c] = create_node();
26            }
27            cur = nxt[cur][c];
28        }
29        if (~match_idx[cur]) return match_idx[cur];
30        return match_idx[cur] = id;
31    }
32
33    void build() {
34        queue<int> q;
35
36        for (int c = 0; c < SIGMA; ++c) {
37            int chi = nxt[0][c];
38            if (chi) {
39                fail_link[chi] = dict_link[chi] = 0;
40                q.push(chi);
41            }
42        }
43
44        while (!q.empty()) {
45            int u = q.front();
46            q.pop();

```

```

47         for (int c = 0; c < SIGMA; ++c) {
48             int chi = nxt[u][c];
49
50             if (chi) {
51                 fail_link[chi] = nxt[fail_link[u]][c];
52
53                 int fail = fail_link[chi];
54                 if (match_idx[fail] != -1)
55                     dict_link[chi] = fail;
56                 else
57                     dict_link[chi] = dict_link[fail];
58                 q.push(chi);
59             } else
60                 nxt[u][c] = nxt[fail_link[u]][c];
61         }
62     }
63 }
64
65 vector<vector<int>> search(const string& text, const vector<int>& lengths) {
66     vector<vector<int>> ret(lengths.size());
67     for (int cur = 0, i = 0; i < text.length(); ++i) {
68         cur = nxt[cur][text[i]];
69
70         for (int u = cur; u; u = dict_link[u]) {
71             int id = match_idx[u];
72             if (id != -1) {
73                 ret[id].push_back(i - lengths[id] + 1);
74             }
75         }
76     }
77     return ret;
78 }
79 }
80 } /* init() insert() build() search() */

```

Dynamic Aho Corasick

```

1 struct AC {
2     int N, P;
3     static const int A = 26;
4     vector<array<int, A> > next;
5     vector<int> link, out_link, cnt;
6     vector<vector<int> > out, rev;
7     AC() : N(0), P(0) { node(); }
8
9     int node() {
10         next.push_back(array<int, A>{0});
11         rev.push_back({});
12         cnt.emplace_back(0);
13         link.emplace_back(0);
14         out_link.emplace_back(0);
15         out.emplace_back();
16         return N++;
17     }
18
19     inline int get(char c) { return c - 'a'; }

```



```

20
21 int add_pattern(const string &T) {
22     int u = 0;
23     for (auto c: T) {
24         if (!next[u][get(c)]) next[u][get(c)] = node();
25         u = next[u][get(c)];
26     }
27     out[u].push_back(P);
28     cnt[u]++;
29     return P++;
30 }
31
32 void build() {
33     queue<int> q;
34     q.push(0);
35     while (!q.empty()) {
36         int u = q.front();
37         q.pop();
38         for (int c = 0; c < A; c++) {
39             int v = next[u][c];
40             if (!v) next[u][c] = next[link[u]][c];
41             else {
42                 link[v] = u ? next[link[u]][c] : 0;
43                 rev[link[v]].push_back(v);
44                 cnt[v] += cnt[link[v]];
45                 out_link[v] = out[link[v]].empty() ?
46                     out_link[link[v]] : link[v];
47                 q.push(v);
48             }
49         }
50     }
51 }
52
53 int advance(int u, char c) {
54     return next[u][get(c)];
55 }
56
57 vector<vector<int> > search(const string &s, vector<int> &pat) {
58     int n = (int) s.length(), m = (int) pat.size();
59     int u = 0;
60     vector<vector<int> > ret(m);
61     for (int i = 0; i < n; i++) {
62         char c = s[i];
63         u = advance(u, c);
64         int x = u;
65         while (x != 0) {
66             for (int v: out[x]) ret[v].push_back(i - pat[v] + 1);
67             x = out_link[x];
68         }
69     }
70     return ret;
71 }
72 };
73
74 struct dynamic_AC {
75     int lg;
76     vector<vector<string> > a;

```

```

77     vector<AC> aho;
78
79     dynamic_AC(int LOG = 20) {
80         lg = LOG;
81         a = vector<vector<string> >(lg);
82         aho = vector<AC>(lg);
83     }
84
85     void insert(const string &s) {
86         vector<string> have = {s};
87         for (int i = 0; i < lg; i++) {
88             if (i == lg - 1) {
89                 lg++;
90                 a.push_back({});
91                 aho.push_back(AC());
92             }
93             if (a[i].empty()) {
94                 swap(have, a[i]);
95                 for (auto &v: a[i]) aho[i].add_pattern(v);
96                 aho[i].build();
97                 break;
98             } else {
99                 have.insert(end(have), begin(a[i]), end(a[i]));
100                 vector<string>().swap(a[i]);
101                 aho[i] = AC();
102             }
103         }
104     }
105
106     ll cnt(const string &s) {
107         ll answer = 0;
108         for (int i = 0; i < lg; i++) {
109             int state = 0;
110             for (auto &v: s) {
111                 state = aho[i].advance(state, v);
112                 answer += aho[i].cnt[state];
113             }
114         }
115         return answer;
116     }
117 };
118
119 struct FullyDynamicAC {
120     dynamic_AC added, removed;
121
122     // Optional: Protects against deleting a string that doesn't exist.
123     // If the problem guarantees valid deletions, you can remove this map.
124     unordered_map<string, int> active_freq;
125
126     void insert(const string& s) {
127         added.insert(s);
128         active_freq[s]++;
129     }
130
131     void erase(const string& s) {
132         if (active_freq[s] > 0) {
133             removed.insert(s);

```

```

134         active_freq[s]--;
135     }
136 }
137
138 long long cnt(const string& s) {
139     return added.cnt(s) - removed.cnt(s);
140 }
141 };

```

Suffix Array

```

1 // O(n log(n))
2 struct suffix {
3     int n;
4     vector<int> p, c, lcp;
5     string s;
6
7     explicit suffix(string _s) : n(int(_s.size()) + 1), s(move(_s)), p(n),
8         c(n), lcp(n - 1) {
9         s += char(0);
10        iota(p.begin(), p.end(), 0);
11        sort(p.begin(), p.end(), [&](int i, int j) { return s[i] < s[j]; });
12        for (int i = 1; i < n; i++) c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
13        vector<int> nc(n), np(n);
14        int k = 1;
15        while (k < n) {
16            vector<int> f(n + 1);
17            for (int i = 0; i < n; i++) p[i] = (p[i] - k + n) % n, f[c[i] + 1]++;
18            for (int i = 1; i <= n; i++) f[i] += f[i - 1];
19            for (int i = 0; i < n; i++) np[f[c[p[i]]]++] = p[i];
20            swap(p, np), nc[p[0]] = 0;
21            for (int i = 1; i < n; i++)
22                nc[p[i]] = nc[p[i - 1]] +
23                    (c[p[i]] != c[p[i - 1]]
24                     || c[(p[i] + k) % n] != c[(p[i - 1] + k) % n]);
25            swap(c, nc), k <= 1;
26        }
27        for (int i = k = 0; i < n - 1; i++) {
28            int j = p[c[i] - 1];
29            for (; s[i + k] == s[j + k]; k++);
30            if (c[i]) lcp[c[i] - 1] = k;
31            k = max(k - 1, 0);
32        }
33        s.pop_back(), c.pop_back(), n--;
34        p.erase(p.begin()), lcp.erase(lcp.begin());
35        for (auto &i : c) i--;
36    }
37    vector<vector<int>> table;
38    void buildLcp() {
39        int LOG = __lg(n) + 1;
40        table.resize(LOG, vector<int>(n));
41        table[0] = lcp;
42        for (int l = 1; l < LOG; l++) {
43            for (int i = 0; i + (1 << (l - 1)) < n; i++) {
44                table[l][i] = min(table[l - 1][i], table[l - 1][i + (1 << (l - 1))]);

```

```

45     }
46 }
47 }
48 int query(int l, int r) { // 0-based
49     if(l == r) return n - 1 - l;
50     l = c[l], r = c[r];
51     if(l > r) swap(l, r);
52     r--;
53     int len = __lg(r - l + 1);
54     return min(table[len][l], table[len][r - (1 << len) + 1]);
55 }
56 auto count(string const &t) { // O(log(n) * size(t))
57     int l = int(lower_bound(p.begin(), p.end(), -1, [&](int i, int j) {
58         return s.substr(i, min<size_t>(t.size(), n - i)) < t;
59     }) - p.begin());
60     int r = int(lower_bound(p.begin() + l, p.end(), -1, [&](int i, int j) {
61         return s.substr(i, min<size_t>(t.size(), n - i)) <= t;
62     }) - p.begin()) - 1;
63     return pair{l, r};
64 }
65 };

```

Suffix Automaton

```

1 namespace sam {
2     const int N = 4000200, SIGMA = 26; // N two times the max length
3     int sa[N][SIGMA], len[N], fail[N], terminal[N], nodes, lst, C[N];
4
5     void init() {
6         nodes = lst = 0;
7         memset(sa[0], -1, sizeof sa[0]);
8         fail[0] = -1;
9         len[0] = terminal[0] = 0;
10    }
11
12    int create_node() {
13        nodes++;
14        memset(sa[nodes], -1, sizeof sa[nodes]);
15        terminal[nodes] = 0;
16        return nodes;
17    }
18
19    void insert(char ch) {
20        int u = create_node();
21        len[u] = len[lst] + 1;
22        int p, q;
23        for (p = lst; ~p; p = fail[p]) {
24            q = sa[p][ch];
25            if (~sa[p][ch]) break;
26            sa[p][ch] = u;
27        }
28
29        if (p == -1) {
30            fail[u] = 0;
31        } else if (len[p] + 1 == len[q]) {
32            fail[u] = q;

```

```

33     } else {
34         int x = create_node();
35         len[x] = len[p] + 1;
36         memcpy(sa[x], sa[q], sizeof sa[0]);
37         for (int v = p; ~v && sa[v][ch] == q; v = fail[v])
38             sa[v][ch] = x;
39         fail[x] = fail[q];
40         fail[q] = fail[u] = x;
41     }
42     lst = u;
43     C[u] = 1;
44 }
45
46 void insert(const string &s) {
47     for (auto ch: s)
48         insert(ch);
49 }
50
51 string walk(const string &s) {
52     int cur = 0;
53     for (auto ch: s) {
54         if (sa[cur][ch] == -1) return "NO"s;
55         cur = sa[cur][ch];
56     }
57     return "YES";
58 }
59
60 void pre_count() {
61     vector<vector<int>> > vlen(N);
62     for (int i = 0; i < N; i++) vlen[len[i]].push_back(i);
63     for (int l = N - 1; ~l; l--)
64         for (auto x: vlen[l])
65             C[fail[x]] += C[x];
66 }
67
68 int count(const string &s) {
69     int cur = 0;
70     for (auto ch: s) {
71         if (!~sa[cur][ch]) return 0;
72         cur = sa[cur][ch];
73     }
74     return C[cur];
75 }
76
77 vector<ll> count_distinct() {
78     vector<ll> dp(nodes + 1), outdeg(nodes + 1);
79     vector<vector<int>> > indeg(nodes + 1);
80     for (int i = 0; i <= nodes; i++)
81         for (int j = 0; j < SIGMA; j++)
82             if (~sa[i][j]) {
83                 outdeg[i]++;
84                 indeg[sa[i][j]].push_back(i);
85             }
86     queue<int> q;
87     for (int i = 0; i <= nodes; i++)
88         if (!outdeg[i])

```

```

89         q.push(i);
90     while (!q.empty()) {
91         int top = q.front();
92         q.pop();
93         dp[top] = 1;
94         for (int i = 0; i < SIGMA; i++)
95             if (~sa[top][i])
96                 dp[top] += dp[sa[top][i]];
97         for (auto x: indeg[top]) {
98             outdeg[x]--;
99             if (!outdeg[x]) q.push(x);
100         }
101     }
102
103     return dp;
104 }
105
106 ll count_distinct2() {
107     ll ret = 0;
108     for (int i = 1; i <= nodes; i++)
109         ret += len[i] - len[fail[i]];
110     return ret;
111 }
112
113 ll sum_distinct() {
114     vector<ll> d(nodes + 1), ans(nodes + 1), outdeg(nodes + 1);
115     vector<vector<int>> indeg(nodes + 1);
116     queue<int> q;
117     for (int i = 0; i <= nodes; i++) {
118         for (int j = 0; j < SIGMA; j++)
119             if (~sa[i][j]) {
120                 outdeg[i]++;
121                 indeg[sa[i][j]].push_back(i);
122             }
123         if (!outdeg[i]) q.push(i);
124     }
125     while (!q.empty()) {
126         int top = q.front();
127         q.pop();
128
129         d[top] = 1;
130         for (int i = 0; i < SIGMA; i++)
131             if (~sa[top][i]) {
132                 d[top] += d[sa[top][i]];
133                 ans[top] += d[sa[top][i]] + ans[sa[top][i]];
134             }
135
136         for (int x: indeg[top])
137             if (--outdeg[x])
138                 q.push(x);
139     }
140     return ans.front();
141 }
142
143 ll sum_distinct2() {
144     ll ret = 0;

```

```

145     for (int i = 1; i <= nodes; i++) {
146         int a = len[i], b = len[fail[i]] + 1;
147         ll num = a-b+1;
148         ret += num * (a+b)/2;
149     }
150     return ret;
151 }
152
153 string get_kth(ll k) {
154     auto d = count_distinct();
155     string ret;
156     int cur = 0;
157     while (k) {
158         for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
159             if (d[sa[cur][i]] >= k) {
160                 k--;
161                 ret.push_back(i);
162                 cur = sa[cur][i];
163                 break;
164             }
165             k -= d[sa[cur][i]];
166         }
167     }
168     return ret;
169 }
170
171 string smallest_shift(int n) {
172     int cur = 0;
173     string ret;
174     while (n--) {
175         for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
176             cur = sa[cur][i];
177             ret.push_back(i);
178             break;
179         }
180     }
181     return ret;
182 }
183
184 vector<int> vis(N);
185 void dfs(int cur, const vector<ll>& d) {
186     if (vis[cur]) return;
187     for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
188         cout << cur << "," << d[cur]-1 << '□' << sa[cur][i] <<
189             "," << d[sa[cur][i]]-1 << '□' << char(i+'a') << '\n';
190         dfs(sa[cur][i], d);
191     }
192 }
193 void prnt() {
194     auto d = count_distinct();
195     dfs(0, d);
196 }
197 } /* init() insert() */

```